

# Requirements Document

Project: **IssueFlow** – Ticket Management Backend Platform

*TDP 2026 Home Assignment*

---

## 1. Project Overview

This assignment challenges you to build a RESTful backend API for **IssueFlow**: a lightweight project and issue tracking platform.

The system manages users, projects, tickets (issues), and comments on tickets.

**You should use AI during the process.** Document how you used the agent and add all relevant files (If used skills, instructions, plan etc.)  
mention the specific model you used to solve this assignment.

You should implement the solution using either:

- **Java 21/25 with Spring Boot 3/4 (skeleton provided)**
- **TypeScript 5.x with NestJS 11(skeleton provided)**

Use the API table in README.md as your implementation contract. Document your setup, build, and run steps in `run.md`.

## 2. Functional Requirements

### 2.1 User Management

The system must support a basic user registry. Each user record acts as the identity behind ticket assignments and comments.

#### Features:

- Register a new user with: `username`, `email`, `full_name`, and `role` (ADMIN or DEVELOPER).
- Fetch a user by their `id`.
- Update a user's details (full name, role).
- Delete a user.
- Fetch all users.

#### Constraints:

- Role must be one of: `ADMIN`, `DEVELOPER`

### 2.2 Authentication

The system must protect all API endpoints using JWT-based authentication.

#### Features:

- `POST /auth/login` - accepts `username` and `password`, returns a signed JWT access token.

- `POST /auth/logout` - invalidates the current token (server-side deny-list or stateless expiry).
- `GET /auth/me` - returns the profile of the currently authenticated user.

## 2.3 Project Management

Projects are the top-level containers that group related tickets together.

### Features:

- Create a new project with: `name`, `description`, and `owner` (a `userId`).
- Fetch a project by `id`.
- Update a project's `name` or `description`.
- Delete a project.
- Fetch all projects.

## 2.4 Ticket Management

Tickets (also called issues) are the core work items tracked in the system. Each ticket belongs to exactly one project.

### Features:

- Create a ticket with: `title`, `description`, `status`, `priority`, `type`, `projectId`, and optional `assigneeId`.
- Fetch a ticket by its `id`.
- Update a ticket's fields (`title`, `description`, `status`, `priority`, `assigneeId`).
- Delete a ticket.
- Fetch all tickets belonging to a given project.
- Update a ticket - A ticket can't be updated simultaneously by two users (or more)

### Constraints:

- Status must be one of: `TODO`, `IN_PROGRESS`, `IN_REVIEW`, `DONE`.
- Priority must be one of: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.
- Type must be one of: `BUG`, `FEATURE`, `TECHNICAL`.
- A ticket can't be updated once it's `DONE`
- A ticket's status may only move forward in the lifecycle: `TODO` → `IN_PROGRESS` → `IN_REVIEW` → `DONE`. Backward transitions are not allowed.

## 2.5 Comment Management

Users can leave comments on tickets to provide context, updates, or discussion.

### Features:

- Add a comment to a ticket with: `content` and `authorId`.
- Fetch all comments for a given ticket.
- Update the content of an existing comment.
- Delete a comment.
- Two users can't edit a comment in the same time (Admin/ Developer)

## 3. Extended Features

### 3.1 Audit log

The system must maintain a persistent, append-only record of all state-changing actions performed within the application. This ensures a transparent history of project and ticket evolutions.

Features:

- All state changing actions should be recorded – those that were manually requested by the user or automatically ran by the system.
- Provide an endpoint to retrieve all logs, or filtered by a specific filed.

### 3.2 Ticket Dependencies

Tickets can depend on other tickets. A ticket cannot transition to DONE if it has unresolved blockers.

Features:

- Add a dependency: `POST /tickets/{ticketId}/dependencies` with body `{ "blockedBy": 42 }` means ticket `ticketId` is blocked by ticket 42.
- List dependencies: `GET /tickets/{ticketId}/dependencies` returns all tickets this ticket is blocked by.
- Remove a dependency: `DELETE /tickets/{ticketId}/dependencies/{blockerId}`.

Constraints:

- Both tickets must exist and belong to the same project.

### 3.3 Attachment Management

Users can attach files to tickets (e.g. screenshots, logs, design assets).

Constraints:

- Maximum file size: `10 MB`. Uploads exceeding this limit must be rejected.
- Allowed file types: `image/png`, `image/jpeg`, `application/pdf`, `text/plain`. Reject all others.

### 3.4 Ticket Export & Import

Support bulk export and import of tickets for project migration, backups, or integrations with external tools.

Features:

- Export tickets: `GET /tickets/export?projectId={id}` returns a CSV file with all tickets for the project. Include fields: `id`, `title`, `description`, `status`, `priority`, `type`, `assigneeId`.
- Import tickets: `POST /tickets/import` accepts a CSV file (`multipart/form-data`) and creates tickets in bulk. The request must specify the target `projectId` as a form field. Returns a summary: `{ "created": 42, "failed": 3, "errors": [...] }`.

Constraints:

- The CSV format must handle commas and quotes inside field values correctly

### 3.5 Soft Delete for Tickets and Projects

Tickets and projects can only be soft-deleted. Soft-deleted records are hidden from standard API responses but can be recovered or audited.

**Features:**

- `GET /tickets/deleted?projectId={id}` and `GET /projects/deleted` list only the soft-deleted records (ADMIN only).
- `POST /tickets/{id}/restore` and `POST /projects/{id}/restore` recover a soft-deleted record (ADMIN only).

### 3.6 @Mention Mechanism in Comments

When a user includes `@username` inside a comment body, the mentioned user is notified and the association is persisted for later retrieval.

**Features:**

- `GET /users/{userId}/mentions` returns all comments where that user was mentioned, newest first.
- Mention metadata is included in each comment response: `mentionedUsers: [{ id, username, fullName }]`.
- On comment update the mention list is re-evaluated: newly added mentions are created, removed mentions are deleted.

**Constraints:**

- Mentions are case-insensitive when matching usernames.

### 3.7 Auto-Scheduling Escalation Level on Tickets

Tickets are automatically escalated in priority when they remain unresolved past a configured due date.

**Features:**

- Ticket creation and update accept an optional `dueDate` field (ISO-8601 datetime).
- For each overdue ticket whose priority is below `CRITICAL`, the priority is promoted one level: `LOW → MEDIUM → HIGH → CRITICAL`.
- When a ticket reaches `CRITICAL` and is still overdue, its `is_overdue` flag is set to true; this flag is visible in all `GET` responses.

**Constraints:**

- Escalation is idempotent: a `CRITICAL` ticket is never escalated further regardless of how far past due it is.
- Escalation only applies to tickets for which `dueDate` has been set.
- A manual priority change by a user (via `PATCH /tickets/{id}`) resets the auto-escalation state for that ticket (`is_overdue` is cleared, and the next escalation cycle re-evaluates from the new priority).
- Escalation does not transition a ticket's status field; only the priority and `is_overdue` flag are modified.

### 3.8 Auto Assignment to Users by Workload

When a ticket is created without an explicit `assigneeId`, the system automatically selects the least-loaded `DEVELOPER` in the project.

**Features:**

- On ticket creation, if `assigneeId` is not provided, the system queries all `DEVELOPER`.
- Workload is defined as the count of non-`DONE` tickets currently assigned to each user within the same project.
- The user with the lowest workload count is auto-assigned. Ties are broken by user registration order (oldest registrant first).
- If no `DEVELOPER` users are linked to the project, the ticket is created with `assigneeId = null` (unassigned) without error.
- `GET /projects/{projectId}/workload` returns a list of `{ userId, username, openTicketCount }` for all users in the project, sorted by `openTicketCount` ascending.
- Each auto-assignment is recorded in the Audit Log with actor = `SYSTEM`, action = `AUTO_ASSIGN`.

**Constraints:**

- Only users with role `DEVELOPER` are candidates for auto-assignment. ADMIN users are excluded.
- Auto-assignment can be overridden at any time by explicitly providing `assigneeId` in a `PATCH /tickets/{id}` request.
- Auto-assignment is not triggered on ticket update, only on creation when `assigneeId` is absent.

## 4. Additional Requirements

### 4.1 Input Validation & Error Handling

- Don't allow invalid values into the API
- In case of error, make sure to return an informative error.

### 4.2 Database & Persistence

- You may use plain SQL or an ORM (e.g., JPA/Hibernate, TypeORM, Prisma).
- Use the provided `compose.yml` to spin up a local PostgreSQL instance via Docker.

### 4.3 Testing

Provide relevant tests covering the key behaviors of your implementation.

- Spring Boot: <https://docs.spring.io/spring-boot/reference/testing/index.html>
- NestJS: <https://docs.nestjs.com/fundamentals/testing>

### 4.4 Documentation

- Please provide a well-documented readme file (`run.md`) with exact steps to: install dependencies, start the database, build the project, run the application, and run the tests.

### 4.5 AI & Agents

- Add main and relevant prompts that shows your interaction with the agents in a `prompts.md` file – **state explicitly which model you used**.
- Add instruction files, skills etc. to your submission.
- Commit all relevant files to your repo.
- Note that you are fully accountable for that code, and be sure to understand it.

## 5. Submission Guidelines

- Once you have completed the assignment, use the HackerRank test invite to submit your Git public repository link. The test will be available for 15 minutes from the moment you start it.
- Please ensure your repo is public and accessible, and all the files have been pushed.
- For any questions/help needed, please contact us here:  
rm-TDPIsrael@intl.att.com
- **You can use the Java with Spring boot or JS/TS with NestJs skeletons project we provided.**
- Use the API description in the readme [README.md](#) to implement the required API's. Make sure to add your setup, build, and run instructions in [run.md](#) file.
- Maven (bundled with mvnw wrapper) or npm